

## Discentes

### ENCONTRO 2: Inserção, Remoção e Gestão de Quantidade

**Objetivo:** Compreender a mecânica de adicionar dados no primeiro espaço vazio e discutir as consequências da remoção em alocações contíguas.

**Instrução:** O Scrum Master tem 15 minutos para guiar o debate lógico. O Arquiteto registra os pseudocódigos ou estratégias de implementação.

### Fatos (Vamos definir os consensos):

*O Scrum Master lê as perguntas. A equipe busca a resposta exata na OAT e o Arquiteto converte em um Fato Inicial no board.*

1. **Regra de Inserção (Create):** Segundo a especificação do motor de gerenciamento (CRUD) da OAT, em qual local exato do array uma nova Peça ou Serviço deve ser adicionada?
  - **Fato Extraído:** A nova Peça ou Serviço deve ser inserida no **primeiro espaço vazio** do array (no índice correspondente à quantidade atual de itens).
  - **Regra de Remoção (Delete):** A OAT sugere duas técnicas para a exclusão de um registro (remoção lógica ou reorganização). Contudo, qual é a técnica que a OAT exige que seja dada preferência para evitar "buracos" no array?
  - **Fato Extraído:** Escolhemos a técnica de **reorganização (shift)**. Ao remover um item da posição **i**, faremos um laço repassando o elemento da posição **i+1** para **i** até o final do array, decrementado a quantidade total para garantir que não haja buracos.
2. **Isolamento Lógico:** Qual é a restrição arquitetural da OAT sobre como toda a lógica de negócio (incluindo inserção e remoção) deve ser implementada no código?
  - **Fato Extraído:** [Anotar o uso obrigatório de métodos estáticos]



### Brainstorm Investigativo (Pesquisem e debatam!)

1. **O Desafio da Inserção (Create):** Precisamos adicionar um novo registro no primeiro espaço vazio do array. Como o sistema vai saber onde está esse "primeiro espaço vazio" sem precisar percorrer o array inteiro toda vez? Precisamos de variáveis auxiliares?

- **Decisão da Equipe:**
  - Criação de uma estrutura de controle (for) para varrer o array e a criação de uma estrutura condicional if para verificar se o espaço está vazio se o espaço estiver vazio será alocado naquele espaço um novo objeto do tipo serviço peça.
2. **A Dor de Cabeça da Remoção (Delete):** Se o nosso array possui vários elementos e removemos o primeiro deles, um "buraco" será criado na memória. Qual das duas técnicas sugeridas pelo Diretor (exclusão lógica ou reorganização), vocês usarão para resolver isso? Qual é o custo computacional da estratégia escolhida?
- **Decisão da Equipe:** Usaremos a Técnica shift para reorganizar os espaços vazios dentro do array
  - Ao remover um espaço, todos os elementos subsequentes são deslocados uma posição para a esquerda, a última posição é limpa e o contador de quantidade é decrementado.
  - **Custo Computacional**
  - O custo é **O(n)** no pior caso (onde n é a quantidade de itens). Ao remover o primeiro elemento, o sistema precisa deslocar os n-1 elementos restantes em uma casa para trás.
  -
3. **Isolamento da Lógica:** Toda a lógica de negócio deve ser implementada através de métodos estáticos. Como serão as assinaturas desses métodos de inserção e remoção, sabendo que os arrays e a variável de quantidade precisam ser manipulados por eles?
- **Decisão da Equipe:** Para garantir a organização e o isolamento da lógica do sistema, a equipe definiu que todas as operações para Peças e Serviços serão implementadas por meio de **métodos estáticos e públicos**.

## Cenários de Teste:

**ID do Teste:** QA-01 - TESTE DE INSERIR PEÇA COM NOME VAZIO

**Responsável (QA):** Matheus Costa

**Data/Hora:** [04/08/2026 - 21:10]

**O que estamos testando? (Contexto):** O sistema deve recusar o cadastro de uma peça quando o campo nome vem vazio.

**Cenário (Ação e Resultado):**

- **Dado que** nome da peça vem vazio
- **Quando** chamo createPeca(1, "", "Bosch", 10, 45.50, 20.00)
- **Então** o sistema deve retornar uma mensagem de erro informando que o nome não pode ser vazio.

**Status:** [ Falhou ]

**Observações do Piloto/Dev:** [Espaço para justificar caso tenha falhado]

**ID do Teste:** QA-02 - TESTE DE INSERÇÃO NO PRIMEIRO ESPAÇO VAZIO

**Responsável (QA):** Matheus Costa

**Data/Hora:** [04/08/2026 - 21:14]

**O que estamos testando? (Contexto):** Validar a regra de inserção travada na OAT.

**Cenário (Ação e Resultado):**

- Dado que o array pecas está vazio
- Quando insiro duas peças em sequência
- Então a primeira ocupa o índice 0 e a segunda o índice 1.

**Status:** [ Passou]

**Observações do Piloto/Dev:** [Espaço para justificar caso tenha falhado]

**ID do Teste:** QA-03 - TESTE DE INSERÇÃO COM ARRAY CHEIO

**Responsável (QA):** Matheus Costa

**Data/Hora:** [04/08/2026 - 21:19]

**O que estamos testando? (Contexto):** Validar o limite de capacidade do array pecas, que possui tamanho fixo 100.

**Cenário (Ação e Resultado):**

- Dado que as 100 posições já estão ocupadas
- Quando tento inserir mais uma peça
- Então o sistema deveria informar que não há espaço disponível.

**Status:** [ Falhou ]

**ID do Teste:** QA-04 - TESTE DE REMOÇÃO COM REORGANIZAÇÃO (SHIFT)

**Responsável (QA):** Matheus Costa

**Data/Hora:** [04/08/2026 - 21:25]

**O que estamos testando? (Contexto):** Validar a regra de remoção sem gerar buracos no array.

**Cenário (Ação e Resultado):**

- Dado que existem 3 peças cadastradas (códigos 1, 2 e 3)
- Quando removo a peça de código 1
- Então a peça 2 passa ao índice 0, a peça 3 ao índice 1, e o último índice fica null.

**Status:** [ Passou]

**Observações do Piloto/Dev:** [Espaço para justificar caso tenha falhado]

**ID do Teste:** QA-05 - TESTE DE REMOÇÃO DE CÓDIGO INEXISTENTE

**Responsável (QA):** Matheus Costa

**Data/Hora:** [04/08/2026 - 21:29]

**O que estamos testando? (Contexto):** Validar o comportamento quando o código buscado não existe.

**Cenário (Ação e Resultado):**

- Dado que não há peça com código 999
- Quando chamo deletePeca(999)
- Então o sistema deveria informar que o código não foi encontrado.

**Status:** [ Falhou ]

**Observações do Piloto/Dev:** [Espaço para justificar caso tenha falhado]

**ID do Teste:** QA-06 - TESTE DE ISOLAMENTO LÓGICO (MÉTODOS ESTÁTICOS E PÚBLICOS)

**Responsável (QA):** Matheus Costa

**Data/Hora:** 11/08/2026 - 21:10

**O que estamos testando? (Contexto):** Validar o isolamento da lógica de CRUD na classe Gerenciador.

**Cenário (Ação e Resultado):**

- Dado que Gerenciador concentra toda a lógica de CRUD
- Quando inspeciono as assinaturas dos métodos
- Então todos devem ser public static.

**Status:** ☒ Passou]

**Observações do Piloto/Dev:** [Espaço para justificar caso tenha falhado]

## Entrega do Encontro 2

### • Código:

```

1  package main;
2
3  import entidades.Pecas;
4  import entidades.Servico;
5  import services.Gerenciador;
6
7  public class Main {
8      public static void main(String[] args) {
9          // Testando as operações do CRUD unificado
10
11          System.out.println("--- Teste de Peças ---");
12          Gerenciador.createPeca(1, "", "Bosch", 10, 45.50, 20.00);
13          Gerenciador.createPeca(2, "Vela de Ignição", "NGK", 4, 30.00, 12.50);
14
15          Gerenciador.readPecas();
16
17          System.out.println("\nAtualizando peça 1...");
18          Gerenciador.updatePeca(1, "Filtro de Óleo Premium", "Bosch", 15, 60.00, 25.00);
19
20          System.out.println("\nBuscando peça 1:");
21          Gerenciador.buscaLinearPeca(1);
22
23          System.out.println("\nDeletando peça 2...");
24          Gerenciador.deletePeca(2);
25          Gerenciador.readPecas();
26
27
28          System.out.println("\n--- Teste de Serviços ---");
29          Gerenciador.createServico(101, "Troca de Óleo", 30, 50.00);
30          Gerenciador.createServico(102, "Alinhamento", 60, 100.00);
31
32          Gerenciador.readServicos();
33
34          System.out.println("\nAtualizando serviço 101...");
35          Gerenciador.updateServico(101, "Troca de Óleo e Filtro", 45, 80.00);
36
37          System.out.println("\nBuscando serviço 101:");
38          Gerenciador.buscaLinearServico(101);
39
40          System.out.println("\nDeletando serviço 102...");
41          Gerenciador.deleteServico(102);
42          Gerenciador.readServicos();
43      }
44  }

```

## Componentes:

Victor Paulo Teixeira da Silva e Silva

Luis Filipe Silva Lima

Guilherme Pereira Barreto

Brendha Maria Costa Seixas

Matheus Santos Costa

Agson Asafe

Os trabalhos tiveram início às 20:05 com a análise e debate da Pergunta 1. Às 20:10, Luis Filipe deu início à escrita do código com base nas instruções previamente estabelecidas.

Em seguida, às 20:12, foram definidas as arrays responsáveis pelo armazenamento das informações principais, incluindo código numérico, descrição do serviço, tempo estimado, preço de venda e custo.

Às 20:15, estabeleceu-se a utilização do modificador de acesso público (**public**) para as estruturas necessárias.

A segunda etapa começou às 20:17 com uma análise conjunta sobre como dar prosseguimento ao algoritmo.

Às 20:23, foi criada a array associada à variável **GerenciarPecas**.

Às 20:54, após enfrentarem diversos entraves técnicos e dificuldades na execução, os integrantes decidiram reiniciar a confecção do código para garantir uma estrutura limpa e funcional.

Neste momento de reestruturação, ocorreu também a consolidação da redistribuição de tarefas em função da saída de Guilherme por motivos externos. Luís retomou a liderança, seguindo rigorosamente o que foi solicitado na OAT, enquanto Victor assumiu por completo a escrita, o registro detalhado da metodologia e a organização final dos arquivos.

No prosseguimento do código, Luis implementou a regra para remoção de itens. A equipe optou pela estratégia de organização por deslocamento (*shift*): ao remover um elemento presente na posição  $i$ , é executado um laço de repetição que repassa o elemento da posição  $i+1$  para a posição  $i$ , e assim sucessivamente até o final da array. Todo o processo metodológico e lógico foi devidamente registrado por Victor para a composição do relatório final.

Matheus Costa trabalhou como QA no teste do sistema e realizou 6 testes para checar as regras de negócio, os limites do array e a estrutura da classe Gerenciador. No total, o sistema teve 50% de aprovação, com 3 testes que passaram e 3 que falharam.

Nos testes que deram certo, o sistema mostrou que a lógica principal está funcionando. O teste QA-02 confirmou que as peças são salvas corretamente nas primeiras posições do array, como o índice 0 e depois o índice 1.

No QA-04, ao apagar uma peça, o sistema reorganizou o array sem deixar espaços vazios no meio. Já no QA-06, foi confirmado que os métodos da classe Gerenciador estão organizados de forma correta, utilizando os modificadores public static. Por outro lado, os testes que falharam mostraram problemas que precisam de correção. No teste

QA-01, o sistema aceitou cadastrar uma peça com o nome em branco, sem fazer a validação do campo.

No QA-03, o sistema permitiu a inserção de novas peças mesmo após atingir o limite máximo de 100 itens do array.

No QA-05, ao tentar apagar um código que não existe no sistema, a aplicação não exibiu a mensagem de erro esperada.